Data store and transport

The document store moves small, structured values between Typst, R, and Python. Use the Calepin store for parameters, labels, counts, and nested configuration. Avoid the store for more complex objects. Prefer ordinary files to save and exchange data frames, model objects, images, and other large or language-specific values.

Value store

Initialize parameters in Typst with `#calepin.store.set`:

````
#calepin.store.set("species", "versicolor")
#calepin.store.set("min_petal_length", 4.5)

```r
#| store-get: ("species", "min_petal_length")
#| echo: false
filtered <- subset(
  iris,
  Species == species & Petal.Length >= min_petal_length
)
cat(nrow(filtered), "rows selected")
```
````

The named values become direct engine bindings only for that chunk. In R the example receives `species` and `min_petal_length`.

```
21 rows selected
```

Project defaults can live in `calepin.toml`:

```
[store]
species = "versicolor"
min_petal_length = 4.5
```

Override them without editing the notebook:

```
[store]
species = "versicolor"
min_petal_length = 4.5
```

Precedence is project `[store]`, then document `calepin.store.set`, then CLI `--set store.*`. Dotted CLI paths update nested mappings.

Calepin persists the completed store with the other generated artifacts under `.calepin/`. Running `calepin clean` removes that cached store by deleting `.calepin/`.

Engine to Engine

A chunk publishes engine variables with `store-set`. A later chunk imports them with `store-get`. Names are identical in the engine and the document store.

````
```r
#| store-set: ("species_levels", "row_count")
#| results: hide
species_levels <- levels(iris$Species)
```
````

````
row_count <- nrow(iris)
```

```python
#| store-get: ("species_levels", "row_count")
#| store-set: python_summary
python_summary = (
    f"{row_count} rows; species: {', '.join(species_levels)}"
)
print(python_summary)
```
````

The R and Python sessions remain persistent, but chunks execute in document order. This makes alternating pipelines such as R → Python → R deterministic.

```
[store]
species = "versicolor"
min_petal_length = 4.5
```

```
calepin compile iris.typ \
  --set store.species=setosa \
  --set store.min_petal_length=1.5
```

Engine to Typst

After the R writer above commits `species_levels`, Typst can read it with `calepin.store.get`:

```
#let levels = calepin.store.get("species_levels", default: ())
The species are #levels.join(", ").
```

The species are setosa, versicolor, virginica.

Typst to Engine

The reverse direction starts with a Typst initializer and names it in the R chunk's `store-get` option:

```
#calepin.store.set("course", "Data Science")

The course is #calepin.inline("r", raw("cat(course)"), store-get: "course",).
```

The course is Data Science.

A Typst dictionary becomes a named list in R and a dictionary in Python:

````
#calepin.store.set("analysis_config", (
  model: "linear",
  include_intercept: true,
  terms: ("petal_length", "petal_width"),
))

```r
#| store-get: analysis_config
stopifnot(is.list(analysis_config))
cat(
  analysis_config$model,
  "model with terms:",
````

````
  paste(analysis_config$terms, collapse = ", ")
)
```

```python
#| store-get: analysis_config
assert isinstance(analysis_config, dict)
print(
    f"{analysis_config['model']} model with terms: "
    + ", ".join(analysis_config["terms"])
)
```
````

```
150 rows; species: setosa, versicolor, virginica
```

Sequences become Python lists and either R atomic vectors or lists, depending on their contents.

Case study

Data transport between an Engine and Typst is a powerful feature that allows us to do complex typsetting very easily. For example, in this example, we use Tab HTML Element to wrap `R` code chunks. We create two named lists, store their names in the store, and define a custom Typst function that loops over names and creates tabs automatically.

````
#import "/.calepin/calepin.typ" as calepin
#show: calepin.document

// The data frames stay in R; Typst receives only their names through the store.
#let r-tabset(list-name, names-key, echo: false) = {
  let names = calepin.store.get(names-key, default: ())
  if names.len() > 0 {
    calepin.elements.tabs[
      #for (index, name) in names.enumerate() [
        #calepin.elements.tab(name, active: index == 0,)[
          #calepin.chunk("r",
            raw("get(" + json.encode(list-name) + ")[[" + json.encode(name) +
"]]",),
            echo: echo,
          )
        ]
      ]
    ]
  }
}

```r
#| echo: false
#| results: hide
#| store-set: (list1names, list2names)
list1 <- list(
  A = data.frame(x = 1:2),
  B = data.frame(x = 1:2, y = 11:12)
)
list2 <- list(
  K = head(iris),
````

```
  Z = head(mtcars)
)
list1names <- names(list1)
list2names <- names(list2)
```

```
#r-tabset("list1", "list1names")

#r-tabset("list2", "list2names")
```

```
linear model with terms: petal_length, petal_width
```

A

```
linear model with terms: petal_length, petal_width
```

B

K

```
  x
1 1
2 2
```

Z

```
  x  y
1 1 11
2 2 12
```

Limitations

Store values may contain:

- `none` / null;
- booleans;
- signed 64-bit integers;
- finite floating-point numbers;
- Unicode strings;
- arrays; and
- dictionaries with string keys.

R and Python exports must use their supported built-in shapes. Unsupported classes and objects fail instead of being silently converted to strings. Store keys are limited to 256 UTF-8 bytes, values to 1 MiB, the complete store to 8 MiB, and nesting to 64 levels.

Only the built-in R and Python engines support `store-get` and `store-set`. Diagram and arbitrary Jupyter engines do not.